



Tecnicatura en Programación a distancia

Ciclo Lectivo 2026

Programación I

Trabajo Práctico Integrador

Alumno: Christian Nicolás Leiva

Alumno: Sol Micaela Ortiz

Tabla de contenido

Marco Teórico e Investigación	3
1.1.	3
1.2.	3
1.3.	4
1.4.	4
1.5.	4
1.6.	5
1.7.	5
2.	5
2.1.	5
2.2.	6
3.	6
4.	6

Carpeta Digital de Documentación

Técnica - TPI Programación 1

Este documento contiene el desarrollo completo del Marco Teórico, Decisiones Técnicas y Estructura Formal del Trabajo Práctico Integrador, diseñado bajo los criterios de evaluación de la rúbrica de la UTN.

Marco Teórico e Investigación

1.1. Archivos de Texto Plano (Formatos CSV)

Un archivo CSV (valores separados por comas) es un formato genérico de almacenamiento de datos estructurados en formato de texto plano. Cada línea del archivo representa un registro completo o fila, mientras que los datos particulares de dicha fila se dividen mediante un delimitador de texto (comúnmente una coma o un punto y coma). Su simpleza y ligereza lo convierten en un estándar industrial para exportar e importar conjuntos de información entre plataformas heterogéneas.

En el presente desarrollo, el archivo CSV actúa como la base de datos persistente externa. Al iniciar el programa, los registros correspondientes a los atributos de los países (nombre, población, superficie y continente) son parseados línea por línea. Para asegurar la robustez requerida por la cátedra, se implementa un bloque de control de excepciones (try-except) que intercepta errores críticos como la ausencia física del archivo (FileNotFoundError) o inconsistencias de casteo de tipos durante la lectura, evitando la finalización abrupta de la aplicación en consola.

1.2. Listas en Python

Las listas en Python son estructuras de datos secuenciales, indexadas, mutables y dinámicas. Esto significa que permiten almacenar colecciones de elementos ordenados bajo un índice numérico base cero, con la flexibilidad de alterar, remover o añadir datos en tiempo de ejecución sin necesidad de declarar previamente una dimensión estática en la memoria volátil del sistema.

Para este proyecto integrador, la lista representa el "dataset o contenedor global". La estructura almacena en su interior la totalidad de los países que son cargados desde la persistencia externa. Al tratarse de una estructura mutable, permite que operaciones del sistema como la "inserción de un nuevo país" se concreten de manera eficiente mediante métodos nativos como `.append()`, actualizando dinámicamente la memoria interna del software.

1.3. Diccionarios

Un diccionario es una estructura de datos asociativa, mutable y no ordenada que almacena elementos bajo un formato explícito de par clave-valor (key-value). A diferencia de las listas, el acceso a un elemento interno no se realiza mediante un índice posicional numérico, sino invocando de forma directa una clave única (generalmente una cadena de caracteres), lo que provee una semántica limpia y una alta eficiencia en la búsqueda interna de datos.

La abstracción conceptual de cada "País" en este programa se modela estrictamente a través de un diccionario de Python. Cada entidad de nuestro dominio se define con la siguiente estructura interna:

```
{  
    "nombre": str,  
    "poblacion": int,  
    "superficie": int,  
    "continente": str  
}
```

Esta decisión de diseño nos permite manipular las propiedades de los países de forma explícita y segura (por ejemplo, acceder a `pais["poblacion"]`) al momento de realizar filtrados algorítmicos o cálculos estadísticos.

1.4. Funciones y Modularización

La modularización es un paradigma de diseño de software que consiste en dividir un sistema complejo en partes lógicas independientes denominadas funciones o módulos. Cada función debe regirse bajo el principio de "responsabilidad única", es decir, debe estar diseñada para resolver una tarea concreta, acotada y bien definida, abstrayendo su lógica interna del resto del programa principal.

El código fuente de este desarrollo rechaza de manera tajante la programación lineal o monolítica. El sistema se subdivide en funciones específicas (por ejemplo, `cargar_datos()`, `ordenar_paises()`, `calcular_estadisticas()`). Esto no solo disminuye drásticamente el acoplamiento del código y facilita la reutilización de rutinas de validación, sino que cumple con el estándar de calidad técnico indispensable para desglosar y defender la lógica algorítmica durante la instancia del video explicativo obligatorio.

1.5. Estructuras Condicionales y Repetitivas

Las estructuras de control determinan el flujo de ejecución de las instrucciones de un programa. Las condicionales (if-elif-else) bifurcan el camino lógico basándose en la evaluación de expresiones booleanas. Las repetitivas o bucles (while y for) permiten la re-ejecución controlada de un bloque de código; el bucle while ejecuta instrucciones mientras una condición sea verdadera, y el bucle for itera secuencialmente sobre los elementos de una colección o estructura de datos.

En el software implementado, un bucle condicionado while garantiza la persistencia activa del menú interactivo en la consola del usuario hasta que se seleccione explícitamente la opción de

salida. Por otro lado, las estructuras for se combinan de manera interna para recorrer la lista global de países. Las estructuras condicionales se aplican de manera exhaustiva en dos niveles: para filtrar países bajo criterios específicos y para validar las entradas de la consola (asegurando que campos numéricos no reciban cadenas de texto o valores negativos).

1.6. Algoritmos de Ordenamiento

Un algoritmo de ordenamiento es un procedimiento de lógica matemática que reorganiza los elementos de un contenedor o lista siguiendo un criterio relacional específico (numérico o alfabético), modificando la disposición posicional de sus índices para establecer un orden estrictamente ascendente o descendente.

El sistema expone un módulo dedicado al ordenamiento dinámico del dataset. Utilizando algoritmos lógicos basados en comparaciones sucesivas (adaptados para trabajar con diccionarios), el usuario de la consola puede reorganizar la lista de países bajo tres propiedades clave: alfabéticamente por el campo "nombre", o numéricamente por los campos de "poblacion" o "superficie", parametrizando la dirección del orden de acuerdo al requerimiento del operador.

1.7. Estadísticas Básicas

Las estadísticas básicas aplicadas a la informática hacen referencia al procesamiento de subconjuntos numéricos mediante operadores aritméticos para extraer conclusiones métricas organizadas, tales como valores extremos (máximos y mínimos) y medidas de tendencia central (promedios aritméticos).

El programa automatiza de forma exacta las siguientes consultas estadísticas solicitadas por el dominio del problema: la detección del país con el índice máximo y mínimo de población (mediante búsquedas lineales comparativas), el cálculo del promedio general de superficie (sumatorio total acumulada dividida la cantidad total de registros) y la tabulación cuantitativa absoluta que determina cuántos países pertenecen a cada continente mapeado.

2. Decisiones Técnicas y Arquitectura

2.1. Estructura de Datos Elegida

La arquitectura lógica del programa se fundamenta en una **Lista de Diccionarios**. Esta estructura compuesta combina las ventajas de indexación y dinamismo de las listas con la semántica descriptiva de los diccionarios.

Almacenar los registros de esta forma optimiza significativamente las operaciones críticas del TPI:

- **Lectura:** Cada fila extraída del CSV se transforma directamente en un objeto diccionario independiente antes de incorporarse al contenedor.
- **Iteración:** Los algoritmos de ordenamiento y filtrado recorren la lista externa e inspeccionan internamente la clave requerida de cada diccionario en tiempo lineal.

2.2. Lógica del Flujo de Consola (Menú)

El sistema se gobierna a través de una interfaz de línea de comandos (CLI) estructurada bajo un bucle infinito controlado. El flujo operativo responde a la siguiente secuencia lógica:

1. El sistema inicializa la lista global invocando la lectura del archivo CSV base.
2. Se despliega de manera visual el menú de opciones en consola.
3. Se captura la opción ingresada por el operador, sometiéndola a un proceso estricto de sanitización y validación de tipos para evitar excepciones en caliente.
4. Mediante una estructura condicional múltiple, se redirige el flujo hacia la función modularizada correspondiente.
5. Tras finalizar la subrutina, el bucle reinicia el flujo visual, permitiendo operaciones continuas hasta la instrucción de cierre de sesión.

3. Conclusiones y Aprendizajes

A lo largo de la cursada de Programación 1, hemos comprendido que la robustez de un sistema no depende solo de la lógica, sino también de cómo anticipa y gestiona sus propias fallas. El correcto manejo de errores en operaciones cualquier código y en códigos con manejo de archivos resulta vital, ya que un simple error del usuario al ingresar un dato, un formato incorrecto o un permiso denegado pueden derivar en la caída total del programa. Implementar estructuras de validaciones previas permite transformar estos posibles errores en eventos controlados, garantizando que el sistema notifique al usuario y continúe operando sin comprometer la integridad de los datos. Esto es determinante para obtener un software confiable y preparado para entornos reales, donde los imprevistos son parte cotidiana de la ejecución.

Por otro lado, la modularización, entendida como la descomposición del problema en funciones o módulos, no solo facilita la legibilidad y el mantenimiento del código, sino que también aísla los posibles puntos de fallo, lo que genera un entorno más confiable. Al limitar la lógica de lectura y escritura en módulos específicos, evitamos que una excepción en el programa de datos provoque un efecto dominó que arrastre a todo el programa.

4. Fuentes Bibliográficas

1. Matthes, E. (2023). *Python Crash Course: A Hands-On, Project-Based Introduction to Programming*. No Starch Press.
2. Python Software Foundation. (2026). *csv — CSV File Reading and Writing*. Python Documentation. <https://docs.python.org/3/library/csv.html>
3. Van Rossum, G., & Drake, F. L. (2009). *The Python Language Reference*. Python Software Foundation.